Mock Exam 2 — Solutions

Lectures 5–8 (Chapters 4–6)

Prof. Dr. Christoph Weisser

Quantitative Research Methods · HSBI

Summer Semester 2026

Overview — problems and points

Problem	Topic	Points
P1	Logistic regression by hand	18
P2	Discriminant analysis and naive Bayes — concepts	12
P3	Confusion matrix and ROC curve	15
P4	Resampling methods (validation set · CV · bootstrap)	18
P5	Model selection with C_p and BIC	12
P6	Ridge regression and the lasso	15
	Total (90 minutes; points $\approx$ minutes)	**90**

How to use this deck

Attempt each problem under exam conditions before looking at the teal boxes. Allowed aids in the exam: a non-programmable calculator and one handwritten A4 sheet of notes (both sides). Round final results to 3 decimals.

Problem 1 — Logistic regression by hand (18 P)

A bank models a credit-card customer's default probability from the monthly balance x (EUR, range 0–400) by logistic regression, fitted in Python with statsmodels (`sm.Logit(y, X).fit()`):

	coef	std err	z	$P > \lvert z \rvert$
`const`	-4.0000	0.400	-10.000	< 0.0001
`balance`	0.0200	0.002	10.000	< 0.0001

(a) Read $\hat{\beta}_0, \hat{\beta}_1$ from the output; predicted default probability at $x = 100$ and $x = 250$. (4 P)

(b) Odds and log-odds at both balances ($x = 100$, $x = 250$); interpret the odds at $x = 250$. (4 P)

(c) Balance x^* with $\hat{p}(x^*) = 0.5$. (3 P)

(d) Odds factor per EUR and per 100 EUR; why is "probability $+0.02$ per EUR" wrong? (4 P)

(e) Two reasons why linear regression on the 0/1 response is inappropriate. (3 P)

Problem 1 — solution (a): predicted probabilities

Solution 1(a) — read the coefficients then substitute

Step 1 — read the fitted model. From the `statsmodels` output, $\hat{\beta}_0 = -4$ (row `const`) and $\hat{\beta}_1 = 0.02$ (row `balance`). The fitted linear predictor (the log-odds) and the probability are

$$\hat{\eta}(x) = \hat{\beta}_0 + \hat{\beta}_1 x = -4 + 0.02\,x, \qquad \hat{p}(x) = \frac{e^{\hat{\eta}(x)}}{1 + e^{\hat{\eta}(x)}}.$$

Step 2 — evaluate the linear predictor. $\hat{\eta}(100) = -4 + 0.02 \cdot 100 = -2$ and $\hat{\eta}(250) = -4 + 0.02 \cdot 250 = 1$.
Step 3 — push through the logistic (sigmoid) function.

$$\hat{p}(100) = \frac{e^{-2}}{1 + e^{-2}} = \frac{0.1353}{1.1353} = 0.119, \qquad \hat{p}(250) = \frac{e^{1}}{1 + e^{1}} = \frac{2.7183}{3.7183} = 0.731.$$

So the estimated default risk is 11.9 % at 100 EUR and 73.1 % at 250 EUR; the sigmoid keeps every probability inside $(0, 1)$.

Problem 1 — solution (b): odds and log-odds

Solution 1(b) — odds and log-odds at both balances

The *log-odds* equal the linear predictor and the *odds* are its exponential:

$$\log \frac{\hat{p}(x)}{1-\hat{p}(x)} = \hat{\eta}(x), \qquad \frac{\hat{p}(x)}{1-\hat{p}(x)} = e^{\hat{\eta}(x)}.$$

	$x = 100$	$x = 250$
log-odds $\hat{\eta}(x)$	-2.000	1.000
odds $e^{\hat{\eta}(x)}$	$e^{-2} = 0.135$	$e^{1} = 2.718$

Cross-check with (a): $\frac{0.119}{0.881} = 0.135$ ✓ $\qquad \frac{0.731}{0.269} = 2.718$ ✓

Interpretation ($x = 250$): the odds are about 2.72 to 1 — default is roughly 2.7 times as probable as non-default for this customer.

Problem 1 — solution (c): the 50:50 balance

Solution 1(c) — solve for the 50:50 balance

A predicted probability of exactly one half means even odds, hence zero log-odds:

$$\hat{p}(x^*) = 0.5 \iff \frac{\hat{p}}{1-\hat{p}} = 1 \iff \hat{\eta}(x^*) = \log 1 = 0.$$

Set the linear predictor to zero and solve for the balance:

$$-4 + 0.02\,x^* = 0 \implies 0.02\,x^* = 4 \implies x^* = \frac{4}{0.02} = 200 \text{ EUR}.$$

In general $x^* = -\hat{\beta}_0/\hat{\beta}_1$.

Decision boundary

At the default threshold 0.5: below 200 EUR the model predicts "no default" as more likely, above 200 EUR "default". So $x^* = 200$ is the classifier's decision boundary.

Solution 1(d) — the slope on the odds scale

Raising the balance by Δ EUR adds $\hat{\beta}_1\Delta$ to the log-odds and hence *multiplies* the odds by $e^{\hat{\beta}_1\Delta}$:

$$\text{per } +1 \text{ EUR: } e^{\hat{\beta}_1} = e^{0.02} = 1.020\ (+2.0\,\%), \qquad \text{per } +100 \text{ EUR: } e^{100\hat{\beta}_1} = e^{2} = 7.389\ (+639\,\%).$$

Why "probability $+0.02$ **per EUR" is wrong.** $\hat{\beta}_1$ is the change in the *log-odds*, not in the probability. The probability slope

$$\frac{\partial \hat{p}}{\partial x} = \hat{\beta}_1\, \hat{p}(x)\big(1 - \hat{p}(x)\big)$$

depends on x: it is largest at $\hat{p} = 0.5$ (here $x = 200$: $0.02 \cdot 0.25 = 0.005$ per EUR) and nearly 0 in the tails — there is no constant "probability effect per EUR".

Problem 1 — solution (e): why not linear regression

Solution 1(e) — two reasons the linear probability model fails

(1) **Out-of-range predictions.** The straight line $\hat{\beta}_0 + \hat{\beta}_1 x$ is unbounded, so for small or large balances it returns "probabilities" below 0 or above 1 — not sensible estimates of $\Pr(Y = 1 \mid x)$. The logistic function is guaranteed to stay in $[0, 1]$.

(2) **Violated error assumptions.** A 0/1 response cannot have normal, homoscedastic errors: its conditional variance $\mathrm{Var}(Y \mid x) = p(x)\big(1 - p(x)\big)$ depends on x (heteroscedasticity), so the usual OLS standard errors and inference are unreliable.

(Bonus: with > 2 classes a numeric coding of Y also imposes an artificial ordering and equal spacing between classes.*)*

Remember

Logistic regression is a *linear model for the log-odds*. Coefficients are additive on the log-odds scale and multiplicative on the odds scale — never on the probability scale.

Problem 2 — restatement

Problem 2 — Discriminant analysis and naive Bayes (12 P)

(a) LDA vs. QDA: the distinguishing covariance assumption; number of covariance matrices estimated with K classes. (4 P)

(b) Bias–variance: when does LDA beat QDA on test data and vice versa? Role of n and of the true class covariances. (3 P)

(c) Key assumption of naive Bayes; why can it help when p is large relative to n? (3 P)

(d) Which classifiers give linear and which give quadratic or more general boundaries? (2 P)

Problem 2 — solution (a): LDA vs. QDA covariance assumption

Solution 2(a) — the distinguishing assumption

Both methods model each class density as multivariate normal, $X \mid Y = k \sim \mathcal{N}(\mu_k, \Sigma_k)$; they differ *only* in what they assume about the covariance matrices:

- **LDA:** a single, *shared* covariance matrix $\Sigma_1 = \cdots = \Sigma_K = \Sigma$. It estimates **one** $p \times p$ matrix — $p(p+1)/2$ covariance parameters.
- **QDA:** a *class-specific* covariance matrix Σ_k for each class. It estimates K matrices — about $K\,p(p+1)/2$ covariance parameters.

Parameter count drives everything

QDA estimates roughly K times as many covariance parameters as LDA. That extra flexibility is exactly what the bias–variance trade-off in part (b) has to weigh.

Problem 2 — solution (b): the bias–variance trade-off

Solution 2(b) — when does each method win?

QDA is the more flexible method (many more parameters): *low bias, high variance*. LDA is more restrictive: *low variance, but biased if the Σ_k truly differ*.

- **LDA tends to win** when n is small (or p large), so reducing variance is crucial *and* the true class covariances are approximately equal — then LDA's shared-Σ assumption costs little bias.
- **QDA tends to win** when n is large, so its many parameters can be estimated reliably *and* the class covariances clearly differ — then LDA's assumption would create substantial bias.

Rule of thumb

Small n / similar $\Sigma_k \Rightarrow$ LDA; large n / differing $\Sigma_k \Rightarrow$ QDA.

Problem 2 — solution (c): naive Bayes

Solution 2(c) — the conditional-independence assumption

Key assumption. Within each class the p predictors are *conditionally independent*, so the joint class density factorises into its marginals:

$$f_k(x) = f_{k1}(x_1)\, f_{k2}(x_2) \cdots f_{kp}(x_p).$$

Why it helps when p is large relative to n. Instead of a full p-dimensional joint density — which needs all the within-class associations, e.g. $p(p+1)/2$ covariance parameters per class — only p *one-dimensional* marginals per class are estimated. This drastic cut in the parameter count sharply lowers the *variance*. When p is large relative to n, that variance saving usually outweighs the bias from (falsely) ignoring dependence — the bias–variance trade-off once more.

Problem 2 — solution (d): shape of the decision boundaries

Solution 2(d) — shapes of the decision boundaries

- **LDA — linear.** With a shared Σ the quadratic term $x^\top \Sigma^{-1} x$ is common to every class and cancels in the pairwise comparison, leaving discriminant functions linear in x.
- **QDA — quadratic.** Each class keeps its own Σ_k, so the quadratic terms do *not* cancel; the boundaries are quadratic curves/surfaces.
- **Naive Bayes — generally non-linear but additive.** The log-odds take the additive form $\sum_j g_j(x_j)$ (no interaction terms). Only in special cases (e.g. Gaussian marginals with class-shared variances) does it reduce to a linear rule.

Problem 3 — restatement

Problem 3 — Confusion matrix and ROC curve (15 P)

Logistic-regression default classifier on $n = 1{,}000$ customers with threshold $\hat{p} > 0.5$; labels $1 =$ default, $0 =$ no default, evaluated with `scikit-learn` (rows = true, columns = predicted, labels in increasing order):

```
confusion_matrix(y_true, y_pred)
array([[855,  45],
       [ 40,  60]])
```

(a) Read TP/FP/FN/TN from the array; accuracy and error rate; sensitivity; specificity; precision; FPR; interpret sensitivity and precision. (7 P)

(b) Effect of lowering the threshold from 0.5 to 0.2 on sensitivity and specificity; business reason for accepting it. (4 P)

(c) What does the ROC curve plot? Meaning of $\text{AUC} = 0.5$ and $\text{AUC} = 1$. (4 P)

Problem 3 — solution (a.1): read the confusion matrix

Solution 3(a) step 1 — identify the four cells

In `scikit-learn` the *rows* are true classes and the *columns* predicted classes, both ordered 0 (“no default”) then 1 (“default”). With “default” (= 1) as the positive class:

		Predicted 0 (no def.)	**Predicted** 1 (default)
True	0 (no def.)	TN = 855	FP = 45
	1 (default)	FN = 40	TP = 60

Row sums: 900 actual non-defaulters and 100 actual defaulters ($= n = 1000$).

Mnemonic

The diagonal (855, 60) is correct; the off-diagonal (45, 40) are the two error types. “False positive” names the *predicted* label — predicted default but truly not.

Problem 3 — solution (a.2): the five metrics

Solution 3(a) step 2 — the five performance metrics

$$
\begin{aligned}
\text{accuracy} &= \frac{\mathrm{TP}+\mathrm{TN}}{n} = \frac{60+855}{1000} = 0.915 \quad (\text{error rate} = 0.085),\\
\text{sensitivity (recall)} &= \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}} = \frac{60}{100} = 0.600,\\
\text{specificity} &= \frac{\mathrm{TN}}{\mathrm{TN}+\mathrm{FP}} = \frac{855}{900} = 0.950,\\
\text{precision} &= \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}} = \frac{60}{105} = 0.571,\\
\mathrm{FPR} &= \frac{\mathrm{FP}}{\mathrm{FP}+\mathrm{TN}} = \frac{45}{900} = 0.050 = 1 - \text{specificity}.
\end{aligned}
$$

Read-off: sensitivity detects only 60 % of the true defaulters; of all flagged customers just 57.1 % (precision) truly default — the rest are false alarms.

Problem 3 — solution (b): lowering the threshold to 0.2

Solution 3(b) — effect on sensitivity and specificity

Lowering the cut-off from 0.5 to 0.2 flags *more* customers as "default" (predict default whenever $\hat{p} > 0.2$):

- Every true defaulter with $0.2 < \hat{p} \leq 0.5$ is now caught $\Rightarrow$ FN $\downarrow$, so **sensitivity increases**.
- But more non-defaulters are also flagged $\Rightarrow$ FP $\uparrow$ (FPR $\uparrow$), so **specificity decreases**. (The overall error rate typically rises, since non-defaulters are the large majority.)

Business reason. The costs are asymmetric: a missed defaulter (FN — a credit loss) is far more expensive than a false alarm (FP — e.g. an unnecessary credit-line review or call). Trading specificity for sensitivity can lower the *expected cost* even if the raw error rate goes up.

Problem 3 — solution (c): the ROC curve and AUC

Solution 3(c) — what the ROC curve shows

The **ROC curve** plots the *true positive rate* (sensitivity, vertical axis) against the *false positive rate* (1−specificity, horizontal axis) as the classification *threshold* is swept over all values from 1 down to 0. Each threshold produces one $(\mathrm{FPR}, \mathrm{TPR})$ point.

- $\mathrm{AUC} = 0.5$: the curve lies on the diagonal — no better than **random guessing**; the score carries no usable ranking information.
- $\mathrm{AUC} = 1$: a **perfect** classifier — some threshold reaches sensitivity $= 1$ and specificity $= 1$ simultaneously (the curve hugs the top-left corner).

Probabilistic reading. $\mathrm{AUC} =$ probability that a randomly chosen defaulter is scored higher than a randomly chosen non-defaulter.

Problem 4 — restatement

Problem 4 — Resampling methods (18 P)

(a) $n = 200$: number of model *fits* for the validation-set approach (50/50 split) · 10-fold CV · LOOCV — with justification. (4 P)

(b) LOOCV vs. 10-fold CV: bias and variance; which is preferred in practice? (4 P)

(c) Show $\Pr(i \in \text{bootstrap sample}) = 1 - (1 - 1/n)^n$; evaluate at $n = 5$; limit as $n \to \infty$. (5 P)

(d) $B = 3$ bootstrap estimates $\hat{\alpha}^* = 0.4$ / 0.5 / 0.6 (toy example; in practice $B \approx 1{,}000$): compute $\widehat{\mathrm{SE}}_B(\hat{\alpha})$; what does it estimate; why is the bootstrap useful? (5 P)

Problem 4 — solution (a): number of model fits ($n = 200$)

Solution 4(a) — how many times is the model refitted?

(i) **Validation set (one 50/50 split): 1 fit.** Train once on the 100 training observations; evaluate once on the 100 held-out ones.

(ii) **10-fold CV: 10 fits.** Split into 10 folds of 20; each fold is held out once while the model is refitted on the remaining 180. 10 folds $\Rightarrow$ 10 fits.

(iii) **LOOCV: 200 fits.** Each single observation is held out once and the model refitted on the other 199; LOOCV $=$ n-fold CV, so $n = 200$ fits. (For least squares a shortcut formula avoids the refits, but generically n fits are needed.)

Cost ordering

Validation set (1) $\ll$ 10-fold (10) $\ll$ LOOCV ($n = 200$): more fits buy a lower-bias error estimate at higher computational cost.

Problem 4 — solution (b): bias and variance of CV

Solution 4(b) — LOOCV vs. 10-fold CV

Bias. LOOCV trains on $n-1$ observations (almost the full sample) $\Rightarrow$ *nearly unbiased* for the true test error. 10-fold CV trains on only $0.9\,n$ observations $\Rightarrow$ it *slightly overestimates* the test error (a little more bias).

Variance. LOOCV has the *higher* variance: its n fitted models use almost identical training sets, so their errors are strongly *positively correlated*, and averaging highly correlated quantities reduces variance only a little. The 10 folds overlap less, giving less-correlated fits and a lower-variance estimate.

Practice

Use $k=5$ or $k=10$: a good bias–variance compromise, and far cheaper than the n refits of LOOCV.

Problem 4 — solution (c): bootstrap inclusion probability

Solution 4(c) — the inclusion probability

A bootstrap sample is n draws *with replacement*. On a single draw, observation i is chosen with probability $1/n$ and missed with probability $1 - \frac{1}{n}$. The n draws are independent, so

$$\Pr(i \notin \text{sample}) = \left(1 - \tfrac{1}{n}\right)^n \implies \Pr(i \in \text{sample}) = 1 - \left(1 - \tfrac{1}{n}\right)^n.$$

At $n = 5$: $1 - \left(\frac{4}{5}\right)^5 = 1 - 0.32768 = 0.672.$ **As $n \to \infty$:** $\left(1 - \frac{1}{n}\right)^n \to e^{-1}$, so $1 - e^{-1} \approx 1 - 0.368 = 0.632.$

Rule of thumb

A bootstrap sample contains about $2/3$ ($\approx 63.2\,\%$) of the *distinct* original observations; the remaining $\approx 37\,\%$ form the "out-of-bag" set.

Problem 4 — solution (d): bootstrap standard error

Solution 4(d) — the bootstrap standard error

Step 1 — bootstrap mean. $\bar{\alpha}^* = (0.4 + 0.5 + 0.6)/3 = 0.5.$

Step 2 — deviations. $0.4 - 0.5 = -0.1, \quad 0.5 - 0.5 = 0, \quad 0.6 - 0.5 = +0.1.$

Step 3 — plug into the formula (with $B - 1 = 2$):

$$\widehat{\mathrm{SE}}_B(\hat{\alpha}) = \sqrt{\frac{(-0.1)^2 + 0^2 + (0.1)^2}{3-1}} = \sqrt{\frac{0.02}{2}} = \sqrt{0.01} = 0.100.$$

What it estimates. The standard deviation of $\hat{\alpha}$ over repeated samples from the population — how much the estimate would vary if the data were collected anew.

Why bootstrap

It resamples the observed data — no closed-form SE formula and no new data needed — and works for almost any statistic ($B \approx 1{,}000$ in practice).

Problem 5 — restatement

Problem 5 — Model selection with Cp and BIC (12 P)

Best subset selection with $n = 100$ and $\hat{\sigma}^2 = 4$ (from the full model):

d	1	2	3	4
RSS_d	500	444	428	424

$$C_p = \tfrac{1}{n}\left(\text{RSS} + 2d\hat{\sigma}^2\right), \qquad \text{BIC} = \tfrac{1}{n}\left(\text{RSS} + \log(n)\, d\, \hat{\sigma}^2\right), \qquad \log(100) \approx 4.605.$$

(a) C_p for all four models. (4 P)

(b) BIC for all four models. (4 P)

(c) Using $C_p = (5.080, 4.600, 4.520, 4.560)$ and BIC$= (5.184, 4.808, 4.833, 4.977)$: which d does each criterion select? Explain the disagreement. (2 P)

(d) Why can training RSS alone never select d? (2 P)

Problem 5 — solution (a): Mallows' C_p

Solution 5(a) — the Cp criterion

Penalty per predictor: $2\hat{\sigma}^2 = 2 \cdot 4 = 8$; with $n = 100$, $C_p = (\text{RSS}_d + 8d)/100$. For example,

$$d = 1: \ (500 + 8 \cdot 1)/100 = 508/100 = 5.080, \qquad d = 3: \ (428 + 8 \cdot 3)/100 = 452/100 = 4.520.$$

Doing this for every d:

d	1	2	3	4
penalty $2d\hat{\sigma}^2 = 8d$	8	16	24	32
$\text{RSS}_d + 8d$	508	460	452	456
C_p	5.080	4.600	**4.520**	4.560

Minimised at d = 3 ($C_p = 4.520$).

Problem 5 — solution (b): BIC

Solution 5(b) — the BIC criterion

Penalty per predictor: $\log(n)\,\hat{\sigma}^2 = \log(100)\cdot 4 = 4.605\cdot 4 = 18.421$; thus BIC $= (\mathrm{RSS}_d + 18.421\,d)/100$. For example,

$$d = 1: \ (500 + 18.421\cdot 1)/100 = 518.421/100 = 5.184.$$

Doing this for every d (the totals carry the full precision of $\log 100 = 4.60517$):

d	1	2	3	4
$\mathrm{RSS}_d + 18.421\,d$	518.421	480.841	483.262	497.683
BIC	5.184	**4.808**	4.833	4.977

Minimised at d = 2 (BIC = 4.808).

Problem 5 — solution (c): which model, and why they disagree

Solution 5(c) — comparing the two penalties

From (a) and (b), $C_p = (5.080, 4.600, 4.520, 4.560)$ and $\text{BIC} = (5.184, 4.808, 4.833, 4.977)$:

$$C_p \text{ is minimised at d} = 3\ (4.520), \qquad \text{BIC is minimised at d} = 2\ (4.808).$$

Both are "RSS + size penalty", but the per-predictor charge differs:

$$C_p:\ 2\hat{\sigma}^2 = 8 \qquad \text{vs.} \qquad \text{BIC}:\ \log(n)\hat{\sigma}^2 = 18.421.$$

Since $\log(100) = 4.605 > 2$, BIC penalises each extra predictor much more heavily and prefers *smaller* models. The RSS drop from $d = 2$ to $d = 3$ is $444 - 428 = 16$: *larger* than the C_p penalty 8 (so C_p accepts predictor 3) but *smaller* than the BIC penalty 18.421 (so BIC rejects it).

Problem 5 — solution (d): why training RSS alone cannot pick d

Solution 5(d) — RSS is monotone in d

Adding a predictor to a nested model can never *increase* the training RSS (at worst its coefficient is set to 0). So RSS decreases *monotonically* in d — equivalently, training R^2 increases monotonically:

$$500 > 444 > 428 > 424.$$

RSS-based selection would therefore *always* pick the largest model $d = 4$, regardless of whether the extra predictors carry real signal: training error is an ever more optimistic estimate of test error as flexibility grows.

The fix

Either penalise model size (C_p, AIC, BIC, adjusted R^2) or estimate the test error directly (validation set, cross-validation).

Problem 6 — Ridge regression and the lasso (15 P)

Linear model with standardised predictors $X_1, \ldots, X_p$.

(a) Objectives of ridge and lasso; which coefficient is not penalised? (4 P)

(b) Behaviour as $\lambda \to 0$ and $\lambda \to \infty$; which Chapter-3 method is recovered as $\lambda \to 0$? (3 P)

(c) Why can the lasso set coefficients exactly to zero (geometry for $p = 2$)? (3 P)

(d) Why standardise the predictors first? (2 P)

(e) Ridge fits at $\lambda = 0.1$ and $\lambda = 100$ (standardised coefficients) and a 10-fold CV grid: which fit has lower bias / lower variance; which λ should be chosen? (3 P)

	$\lambda = 0.1$	$\lambda = 100$
$\hat{\beta}_1$	0.92	0.31
$\hat{\beta}_2$	0.85	0.29
$\hat{\beta}_3$	-0.41	-0.12
$\hat{\beta}_4$	0.20	0.06

λ	0.1	1	10	100
CV error	12.8	11.9	10.6	13.4

Problem 6 — solution (a): the two objectives

Solution 6(a) — ridge vs. lasso penalties

With $\text{RSS} = \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij}\right)^2$ and tuning parameter $\lambda \geq 0$:

$$\text{Ridge:} \quad \min_{\beta} \ \text{RSS} + \lambda \sum_{j=1}^{p} \beta_j^2 \ \left(= \text{RSS} + \lambda \|\beta\|_2^2\right),$$

$$\text{Lasso:} \quad \min_{\beta} \ \text{RSS} + \lambda \sum_{j=1}^{p} |\beta_j| \ \left(= \text{RSS} + \lambda \|\beta\|_1\right).$$

Ridge uses the squared (ℓ_2) penalty, the lasso the absolute (ℓ_1) penalty. The intercept β_0 is *not* penalised — shrinking the overall level of y would make no sense.

Problem 6 — solution (b): limiting behaviour of λ

Solution 6(b) — the two extremes

- $\boldsymbol{\lambda} \to 0$: the penalty vanishes and both estimators approach the *ordinary least-squares* (OLS) fit of Chapter 3.
- $\boldsymbol{\lambda} \to \boldsymbol{\infty}$: the penalty dominates and all penalised coefficients are forced to 0 — the null model with intercept only.

In between, the two behave differently: ridge shrinks coefficients *towards* zero (exactly zero only in the limit), whereas the lasso sets more and more coefficients *exactly* to zero as λ grows. Increasing λ traces a whole path of fits whose flexibility *decreases* continuously.

Recovered method

As $\lambda \to 0$ both penalised methods reduce to **ordinary least squares**.

Problem 6 — solution (c): why the lasso gives exact zeros

Solution 6(c) — constraint-region geometry

Each penalised problem is equivalent to minimising RSS subject to a *budget* constraint on the coefficients:

$$\text{lasso: } |\beta_1| + |\beta_2| \leq s \quad (\text{a } \textit{diamond}), \qquad \text{ridge: } \beta_1^2 + \beta_2^2 \leq s \quad (\text{a } \textit{disk}).$$

The RSS has elliptical contours centred at the OLS estimate; the solution is where these ellipses first *touch* the constraint region.

- The lasso diamond has sharp **corners on the axes**. Ellipses typically touch a corner — where one coefficient is exactly 0 (sparsity / variable selection).
- The ridge disk has **no corners**, so the contact point generically has *all* coordinates non-zero: ridge shrinks but keeps every predictor.

Problem 6 — solution (d): why standardise the predictors

Solution 6(d) — units matter for penalised fits

OLS is *scale-equivariant*: rescaling X_j (e.g. EUR $\rightarrow$ thousand EUR) simply rescales $\hat{\beta}_j$ and leaves the fit unchanged. The penalties, however, depend on the coefficient *magnitudes* — $\lambda \sum \beta_j^2$ resp. $\lambda \sum |\beta_j|$ — and hence on the units: a predictor measured in small units has a large coefficient and would be shrunk more strongly for a purely *cosmetic* reason. Standardising each X_j to standard deviation 1 puts all coefficients on a common scale, so the penalty acts comparably on every predictor and the solution is independent of the units.

Problem 6 — solution (e): bias–variance and choosing λ

Solution 6(e) — read the two fits and the CV grid

Bias–variance of the two fits. $\lambda = 0.1$ applies almost no shrinkage — close to OLS, more flexible $\Rightarrow$ **lower bias, higher variance**. $\lambda = 100$ shrinks each coefficient to roughly a third of its size (e.g. $0.92 \to 0.31$, $0.85 \to 0.29$; ratios ≈ 0.30–0.34) — a more rigid fit $\Rightarrow$ **higher bias, lower variance**. This is the Chapter-2 trade-off controlled by the single knob λ.

Choosing λ. Use the smallest *cross-validated* error, not the training fit:

$$\lambda = 10: \; 10.6 \; < \; 11.9 \; (\lambda{=}1) \; < \; 12.8 \; (\lambda{=}0.1) \; < \; 13.4 \; (\lambda{=}100).$$

So choose $\boldsymbol{\lambda} = 10$ — intermediate shrinkage; *neither* fit in the coefficient table is optimal.

Big picture for the exam

Regularisation trades a little bias for a large variance reduction; λ is always chosen by cross-validation — never by looking at training RSS.